

Exception Handling in Java

A Beginner's Guide

Zadid Bin Azad

Department of Computer Science and Engineering
Islamic University of Technology
zadidbinazad@iut-dhaka.edu

July 12, 2026

Teaching Insight

This handout is the companion to Lab 9 (*TakaLink*). That lab asks you to signal every failed transaction with a *checked exception* and to handle those failures so the nightly batch survives them. If the words “checked exception,” “throws clause,” or “catch-or-declare” are new to you, read this first. A few boxes titled **Question to Carry Forward** are left unanswered on the page — attempt them yourself; the reasoning in that gap is the point.

Contents

1	Introduction: You Have Already Met Exceptions	1
2	The Problem Exceptions Solve: Return Codes Rot	2
3	Throwing: Making an Exception Travel	2
4	Catching: Dealing With a Travelling Exception	3
4.1	Catching several kinds	3
5	The Exception Family Tree	4
6	Checked Exceptions and the throws Clause	4
7	Writing Your Own Exceptions	5
8	finally: Code That Always Runs	6
9	Putting It Together: A Batch That Survives Failure	6
10	Fail Without Side Effects (the Atomicity Rule)	7
11	Habits of Good Exception Handling	8
12	One-Page Reference	9

1 Introduction: You Have Already Met Exceptions

You have almost certainly seen a program stop with a message like this:

```
1 Exception in thread "main" java.lang.NullPointerException
2   at Wallet.debit(Wallet.java:14)
3   at Bank.settle(Bank.java:31)
4   at Main.main(Main.java:8)
```

That is an *exception*. An exception is Java's way of saying: “**something happened here that I cannot deal with locally, so I am stopping this line of work and reporting it upward.**” The block of lines under the message — `at Wallet.debit ...at Bank.settle ...at Main.main` — is the *stack trace*: it is the trail of method calls that were in progress, printed innermost-first, so you can see *where* the trouble started and *who* had asked for it.

The single most important idea in this whole handout is that **an exception travels**. It is not just a printed message; it is an *object* that leaves the method where the trouble happened and moves *up the chain of callers*, looking for someone willing to deal with it. If nobody deals with it, it reaches the top and the program stops. Everything else here is detail on that one idea: how to *throw* such an object, how it *travels*, and how to *catch* it.

Question for the Class

In the trace above, `main` called `settle`, which called `debit`, where the trouble happened. The message appeared anyway, at the top. What carried the news from `debit` all the way back up to `main` — given that `debit` never returned a value to say “I failed”?

2 The Problem Exceptions Solve: Return Codes Rot

Before exceptions, the only way a method could report trouble was to *return* a special value — a -1, a `null`, a status number. Here is the style, drawn straight from the broken engine in your lab:

```
1 // 0 = OK, 1 = insufficient, 2 = over limit, 3 = bad PIN, 4 = frozen
2 int process(Transaction t) {
3     if (from.frozen) return 4;
4     if (!from.pin.equals(t.pin)) return 3;
5     // ... move the money ...
6     return 0;
7 }
```

Listing 1: Signalling failure with a status code — the disease.

Three things go wrong with this, and they are exactly the pains Lab 9 asks you to cure:

1. **The caller can forget to check.** Nothing forces anyone to read the returned `int`. Write `process(t)`; on a line by itself, ignore the number, and the program marches on as if the money moved — when it did not.
2. **The number carries no meaning and no detail.** “2” means over-limit *today*; next release it means something else. It cannot carry *which* wallet, *how much* it was short, or a *human-readable reason*. It is a bare integer standing in for a whole story.
3. **Good return values and error return values collide.** What if a method genuinely needs to return an `int` — a balance, a count? Now some `ints` mean answers and some mean errors, and only a comment tells them apart.

Working It Through

An exception fixes all three. It *cannot* be silently ignored the way a return value can (the program stops if nobody handles it). It is a full *object*, so it carries a message and any data you attach. And it travels on a completely *separate channel* from the normal return value, so “the answer” and “the failure” never collide.

3 Throwing: Making an Exception Travel

You start an exception on its journey with the `throw` keyword. You throw an *object* — specifically an object of a class that descends from `Throwable` (almost always a subclass of `Exception`). Java gives you many ready-made ones. Here is the smallest useful example:

```
1 public void debit(double amount) {
2     if (amount <= 0) {
3         throw new IllegalArgumentException("amount must be positive, got " +
4             amount);
5     }
6     if (amount > balance) {
7         throw new IllegalStateException("balance " + balance + " too low for "
8             + amount);
9     }
10    balance -= amount;    // only reached if neither check threw
11 }
```

Listing 2: Throwing a built-in exception.

Read `throw new IllegalArgumentException(...)` as one action: *build* an exception object (with `new`, passing it a message) and *launch* it. The instant a `throw` runs, the rest of the method is abandoned — `balance -= amount` on the last line never executes if either check fired. Control leaves `debit` immediately and starts travelling up to whoever called it.

Teaching Insight

Notice what `throw` buys you for free: **the code after a failed check does not run**. You do not need an `else`. The method “falls through” to the real work *only* when every guard above it has passed. This is why validation-by-throwing reads so cleanly: a stack of guards at the top, then the real work at the bottom, no nesting.

4 Catching: Dealing With a Travelling Exception

An exception travelling up the call chain is looking for a `try/catch` that names its type. That is where it stops.

```
1 try {
2     wallet.debit(2037.0);           // this might throw
3     System.out.println("settled"); // skipped if debit throws
4 } catch (IllegalStateException e) {
5     System.out.println("refused: " + e.getMessage());
6 }
7 // execution continues here whether or not it threw
```

Listing 3: The shape of `try/catch`.

The rules of the dance:

- Code inside `try` runs normally until something throws.
- If a `throw` happens, Java *immediately* stops the `try` block — any lines after the throwing call are skipped — and looks at the `catch` clauses.
- If a `catch` names the thrown exception's type (or a parent of it), that block runs. The caught object is available as the variable `e`; `e.getMessage()` gives the message you passed to the constructor.
- After the `catch` finishes, execution continues *below* the whole `try/catch`. The exception has been *handled*; it stops travelling.
- If *no* `catch` matches, the exception keeps travelling up to *this* method's caller, and the search continues there.

4.1 Catching several kinds

A `try` may be followed by several `catch` blocks. Java tries them top to bottom and runs the *first* one that matches.

```

1 try {
2     transaction.settleAgainst(from, to);
3 } catch (InsufficientBalanceException e) {
4     report.reject(transaction, "not enough funds: " + e.getMessage());
5 } catch (FrozenAccountException e) {
6     report.reject(transaction, "account frozen: " + e.getMessage());
7 } catch (TransactionException e) { // parent: any other refusal
8     report.reject(transaction, "refused: " + e.getMessage());
9 }

```

Listing 4: Multiple catch blocks, most specific first.

Where Students Typically Slip

Order matters. A `catch` for a *parent* type will grab a child before the child's own `catch` is reached. If you put `catch (TransactionException e)` *first*, the two specific blocks below it become unreachable and the compiler rejects the code. **Always list the most specific exceptions first and the general parent last.**

5 The Exception Family Tree

Every exception in Java is an object in one inheritance tree. You need to recognise four names in it:

<code>Throwable</code>	the root of everything that can be thrown
↳ <code>Error</code>	serious JVM problems you are not meant to catch (out of memory)
↳ <code>Exception</code>	the family <i>you</i> work with
↳ <code>RuntimeException</code>	the “unchecked” branch

The one split that matters for this course lives inside `Exception`: is a given exception **checked** or **unchecked**?

- An **unchecked** exception is any subclass of `RuntimeException` (for example `NullPointerException`, `IllegalArgumentException`, `IllegalStateException`, `ArrayIndexOutOfBoundsException`).

The compiler does *not* force you to deal with these. They usually mean a *programming bug* — something that should never happen if the code is correct.

- A **checked** exception is any subclass of `Exception` that is *not* under `RuntimeException`. The compiler **forces** every caller to deal with these (you will see how in the next section). They usually mean an *expected, recoverable* situation that a correct program must still plan for — a file that is missing, a network that is down, or *a transaction the business refuses*.

Teaching Insight

The rule of thumb the industry uses: **unchecked** for “the programmer made a mistake” (a bad argument, a null where there should not be one); **checked** for “the world did something the program must be ready for.” A wrong PIN or an insufficient balance is not a bug in your code — it is a normal outcome the switch must handle every night. That is why Lab 9 asks for *checked* exceptions: it makes handling them non-optional.

6 Checked Exceptions and the throws Clause

When a method can throw a *checked* exception and does not catch it itself, it must *announce* that fact in its signature with a **throws** clause. This is a promise to callers: “calling me might hand you back this kind of failure.”

```

1 // "settling this transaction might be refused" -- announced in the signature
2 public void settleAgainst(Wallet from, Wallet to) throws TransactionException {
3     if (from.isFrozen()) throw new FrozenAccountException(from.id());
4     ;
5     if (!from.verifyPin(pin)) throw new InvalidPinException(from.id());
6     if (from.balance() < amount + fee())
7         throw new InsufficientBalanceException(from.id(), amount + fee());
8     // only now, with every check passed, does money move:
9     from.debit(amount + fee());
10    to.credit(amount);
11 }

```

Listing 5: Declaring a checked exception with throws.

Because `settleAgainst` declares **throws** `TransactionException`, the compiler now imposes the **catch-or-declare** rule on anyone who calls it. A caller has exactly two legal choices:

1. **Catch** it — wrap the call in a `try/catch` (this is what the batch loop does), *or*
2. **Declare** it — add **throws** `TransactionException` to *its own* signature, passing the obligation further up.

There is no third option. You cannot silently ignore a checked exception; the code will not compile. *That is the entire point* — it is the compiler guaranteeing that “this might be refused” can never be forgotten.

Question for the Class

The status-code version (`int process(...)`) let a caller write `process(t)`; and ignore the result. The checked-exception version makes that impossible to compile. In one sentence: what did we gain by moving the failure from the *return value* to a *thrown checked exception*?

7 Writing Your Own Exceptions

Built-in exceptions are fine for generic problems, but a domain reads better when its failures have their *own* names. Making one is ordinary inheritance: extend `Exception` for a *checked* exception, or `RuntimeException` for an *unchecked* one.

```
1 // The common parent: "a transaction was refused." Checked, so it extends
  Exception.
2 public class TransactionException extends Exception {
3     public TransactionException(String message) {
4         super(message);           // hand the message up to Exception
5     }
6 }
7
8 // A specific reason. It IS-A TransactionException, so one catch can hold them
  all.
9 public class InsufficientBalanceException extends TransactionException {
10     private final double shortfall;
11
12     public InsufficientBalanceException(String walletId, double needed) {
13         super("wallet " + walletId + " needs " + needed);
14         this.shortfall = needed;
15     }
16     public double shortfall() { return shortfall; } // carry data the handler
17 } // can use
```

Listing 6: A small checked-exception family for a payment switch.

Two design points worth saying aloud:

- **A common parent means one catch can hold the whole family.** Because `InsufficientBalanceException`, `FrozenAccountException`, and the rest all extend `TransactionException`, a single catch (`TransactionException e`) in the batch loop catches *any* refusal — yet you can still write a more specific catch above it when you want to treat one kind differently. This is polymorphism, working on exceptions exactly as it works on any other class hierarchy.
- **An exception can carry data.** The `shortfall` field above lets the handler report *how much* the wallet was short — something a bare status code could never do.

Question to Carry Forward (Q1)

Lab 9 also wants a malformed `Transaction` (a negative amount, a null wallet) to be *impossible to construct*. Should that failure be a *checked* exception under `TransactionException`, or an *unchecked* `IllegalArgumentException` thrown from the constructor? Clue: is a negative amount a normal outcome the nightly run must survive, or a bug in whoever built the object?

8 finally: Code That Always Runs

Sometimes you have cleanup that must happen *whether or not* the `try` threw — closing a file, releasing a lock. A `finally` block runs on every exit path: normal completion, a caught exception, even an exception that is about to keep travelling.

```
1 lock.acquire();
2 try {
```

```

3     doRiskyWork();           // may throw
4 } finally {
5     lock.release();          // runs even if doRiskyWork() throws
6 }

```

Listing 7: finally runs no matter what.

For the specific job of closing resources, modern Java offers *try-with-resources*, which calls `close()` for you automatically:

```

1 try (var reader = new BufferedReader(new FileReader("batch.csv"))) {
2     return reader.readLine();
3 } // reader.close() is called for you, even on an exception

```

Listing 8: try-with-resources closes the reader automatically.

Teaching Insight

You will not need `finally` for the core of Lab 9 — there are no files or locks to release in the settlement logic. It is here so the picture is complete: `try` runs the work, `catch` handles failure, `finally` cleans up regardless.

9 Putting It Together: A Batch That Survives Failure

Now the whole shape Lab 9 is built around. One transaction throws when it is refused; the loop that settles the batch *catches* each refusal, records it, and *keeps going*. One bad transaction never stops the good ones behind it.

```

1 public SettlementReport settleAll() {
2     SettlementReport report = new SettlementReport();
3     for (Transaction t : batch) {
4         try {
5             t.settleAgainst(walletOf(t.fromId()), walletOf(t.toId()));
6             report.recordSettled(t);           // reached only if no throw
7         } catch (TransactionException e) {
8             report.recordRejected(t, e);       // any refusal lands here...
9         }
10        // ...and the loop moves on to the next transaction either way
11    }
12    return report;
13 }

```

Listing 9: The batch loop: catch each failure, continue, report.

Trace one pass with your finger. If `settleAgainst` completes, `recordSettled` runs and the loop advances. If it throws — say an `InsufficientBalanceException` — the `try` is abandoned at the throwing line (so `recordSettled` is *skipped*), the `catch` records the rejection with its reason, and the loop advances anyway. Every transaction gets exactly one outcome: settled, or rejected-with-a-reason. The batch always finishes.

Question for the Class

Suppose the fifth transaction in a batch of a thousand throws. In the *status-code* design, what was the risk to the other 995? In this `try/catch` design, what happens to them instead?

10 Fail Without Side Effects (the Atomicity Rule)

There is one more rule that exceptions make possible and that Lab 9 marks you on. Look again at `settleAgainst`: *every check is done before any money moves*. That ordering is deliberate.

```
1 if (from.isFrozen())          throw new FrozenAccountException(from.id());
2 if (!from.verifyPin(pin)) throw new InvalidPinException(from.id());
3 if (from.balance() < amount + fee())
4     throw new InsufficientBalanceException(from.id(), amount + fee());
5 from.debit(amount + fee());    // no throw can happen after this point
6 to.credit(amount);
```

Listing 10: Right: validate everything, THEN move money.

Where Students Typically Slip

The contractor's engine did the opposite — it debited the sender and *then* checked the limit:

```
1 from.balance -= amount;                // money leaves NOW
2 if (from.spentToday + amount > limit) return 2; // ...too late; already
   gone
```

When that check fails, the money has already left and nothing puts it back. The account is left in a *broken* state by a *failed* operation. This is the single worst bug a payment system can have.

Working It Through

The cure is an ordering discipline: **throw before you touch anything**. Put every check that can fail *above* the first line that changes a balance. Then a thrown exception is guaranteed to leave every wallet exactly as it was — the operation either happens completely or not at all. Because exceptions abandon the rest of the method the instant they are thrown, this ordering is all you need; you do not have to “undo” anything, because nothing was done yet.

11 Habits of Good Exception Handling

- **Catch what you can actually handle.** A catch block should *do* something — record it, retry, translate it. If you have no plan for a failure here, let it travel up to someone who does (declare it with `throws`).
- **Never swallow an exception silently.** An empty catch `{ }` throws away the news that something failed and is almost always a bug. At the very least, record it.
- **Catch narrowly.** Prefer `catch (InsufficientBalanceException e)` over `catch (Exception e)`. A wide catch will also swallow bugs you never meant to hide (a stray `NullPointerException`).
- **Do not use exceptions for ordinary control flow.** Throwing is for the *exceptional* — the refused, the malformed. Do not throw to break out of a loop over normal data; that is what `break` and `return` are for.
- **Put a real message in.** `throw new InsufficientBalanceException("wallet 07xx needs 2037.0")` helps the next person; `throw new InsufficientBalanceException("error")` does not.

- **Validate at the boundary; fail fast.** Reject a bad argument the moment it arrives (in the constructor), so a broken object never exists to cause a confusing failure ten methods later.

Where Students Typically Slip

Two traps beginners hit repeatedly: (1) catching `Exception` everywhere “to be safe,” which hides real bugs; and (2) catching an exception, printing it, and then *continuing as though nothing failed* — so the caller still believes the work succeeded. If you catch and cannot recover, either genuinely recover, or re-throw.

12 One-Page Reference

Keyword / idea	What it does
<code>throw new X("msg")</code>	Builds an exception object and launches it up the call chain; abandons the rest of the current method at once.
<code>try { ... }</code>	Marks code that might throw.
<code>catch (X e) { ... }</code>	Stops an exception of type <code>X</code> (or a subtype) travelling; <code>e</code> is the caught object.
<code>finally { ... }</code>	Runs on <i>every</i> exit path from the <code>try</code> — cleanup that must always happen.
<code>throws X</code> (in signature)	Announces that this method may throw checked exception <code>X</code> ; forces callers to catch-or-declare.
Checked	Subclass of <code>Exception</code> (not <code>RuntimeException</code>). Compiler forces handling. Use for expected, recoverable failures (a refused transaction).
Unchecked	Subclass of <code>RuntimeException</code> . Handling optional. Use for programming bugs (a bad argument): e.g. <code>IllegalArgumentException</code> .
<code>e.getMessage()</code>	The text you passed to the exception's constructor.
Custom exception	<code>class MyException extends Exception</code> (checked) or <code>extends RuntimeException</code> (unchecked); a common parent lets one catch hold a whole family.
Atomicity rule	Put every failing check <i>above</i> the first line that mutates state, so a throw leaves everything untouched.

Teaching Insight

If you remember only one sentence from this handout, make it this: *an exception is an object that travels up the call chain until someone catches it* — and a *checked* exception is one the compiler will not let a caller ignore. Everything Lab 9 asks of you follows from taking those two facts seriously.